

Programación de Sistemas

Unidad 1: Introducción a la Programación de Sistemas y los Compiladores

Nota de clase 02: Estructura y fases de un compilador

Manuel Soto Romero

Araceli Liliana Reyes Cabello

Semestre 2026-2

Colegio de Ciencia y Tecnología UACM

En la nota anterior ubicamos al compilador entre las herramientas de la programación de sistemas. Visto desde fuera, recibe un programa escrito en un lenguaje fuente y produce un programa equivalente en un lenguaje destino. Esa descripción es correcta, pero oculta el trabajo interno. Un compilador debe reconocer unidades elementales, descubrir la estructura del programa, comprobar restricciones de significado y transformar varias representaciones antes de producir su salida.

En esta nota abriremos esa “caja” para estudiar su organización general. Primero distinguiremos las dos grandes etapas de análisis y síntesis. Después seguiremos las fases léxica, sintáctica y semántica; ubicaremos las representaciones intermedias, la optimización y la generación de código; y, finalmente, observaremos cómo se comunican las fases. Se trata de un mapa inicial: cada componente se estudiará con mayor detalle en las unidades posteriores.

Ruta de lectura (20–25 minutos)

Lectura esencial: sigue el fragmento inicial a través del esquema análisis–síntesis; compara las entradas, tareas y salidas de las fases léxica, sintáctica y semántica; identifica la función de la representación intermedia, la optimización y la generación de código; y completa el mini-checkpoint sobre los contratos entre fases.

Lectura complementaria: revisa en una segunda pasada las variantes posibles de arquitectura, los matices sobre cuántas representaciones u optimizaciones puede emplear un compilador y las explicaciones ampliadas sobre información compartida y manejo de errores.

Caso inicial. Un mismo programa, varias representaciones

Consideremos el siguiente fragmento ilustrativo:

```
valor = valor + inc;
```

El fragmento es legible para una persona que reconoce una asignación y una suma. Sin embargo, el compilador no pasa directamente de ese texto al código destino. A lo largo del proceso puede observarlo como una secuencia de caracteres, una secuencia de componentes, una estructura en forma de árbol y una secuencia de operaciones más cercana al lenguaje destino.

El ejemplo procede de los materiales de estructura de compiladores disponibles para el curso. Se emplea únicamente para reconocer las fases: **no establece todavía la sintaxis oficial de IMP.**

1. Del código fuente al código destino: análisis y síntesis

El punto de partida es un **programa fuente**: un texto escrito de acuerdo con las reglas de un lenguaje. El punto de llegada es un **programa destino**: otra representación del programa en un lenguaje diferente.

El lenguaje destino puede ser lenguaje de máquina, ensamblador, un lenguaje intermedio o incluso otro lenguaje de alto nivel. En este curso construiremos en notas y clase el compilador de IMP y extenderemos ese mismo producto en las prácticas para traducir IMP++; ambos recorridos producirán programas en C.

Entre los dos extremos se conserva una exigencia central: el programa destino debe corresponder al significado del programa fuente. Cambiar la representación no autoriza al compilador a cambiar lo que el programa debe hacer.

Definición 1. (Fase de compilación)

Una **fase de compilación** es una parte conceptualmente diferenciada del proceso que recibe una representación del programa, realiza una tarea específica y produce información o una nueva representación para las fases siguientes.

Una primera división organiza el proceso en dos grandes etapas:

- ★ **Análisis.** Examina el programa fuente para reconocer sus componentes, su estructura y las restricciones que debe satisfacer. Su resultado es una representación interna adecuada para continuar la traducción.
- ★ **Síntesis.** Toma la representación obtenida por el análisis y construye una representación equivalente en el lenguaje destino.



La figura muestra una separación conceptual, no una única arquitectura obligatoria. Un compilador real puede usar varias representaciones internas, aplicar varias transformaciones o combinar parte del trabajo de dos fases. Para comprenderlo y construirlo, conviene comenzar con responsabilidades separadas y con entradas y salidas explícitas.

También es necesario distinguir al compilador de la cadena de herramientas que rodea la compilación. En un entorno habitual pueden intervenir un preprocesador, el compilador propiamente dicho, un ensamblador y un ligador. El compilador no realiza necesariamente todas esas tareas.

Observación 1. El destino del compilador del curso es C

En este curso, el producto de nuestra traducción será un programa en C. Después, un compilador de C como GCC o Clang podrá convertir ese programa en una forma ejecutable. Por tanto, generar C completa la traducción prevista para nuestro compilador, pero no toda la cadena que lleva hasta la ejecución en la máquina.

2. Análisis léxico, sintáctico y semántico

El análisis suele dividirse en tres fases. Cada una responde una pregunta distinta y trabaja con el resultado producido por la anterior:

Fase	Pregunta principal	Transformación general
Análisis léxico	¿Cuáles son las unidades elementales del texto?	De una secuencia de caracteres a una secuencia de componentes léxicos o tokens.
Análisis sintáctico	¿Cómo se agrupan esas unidades de acuerdo con la gramática?	De una secuencia de tokens a una representación estructurada del programa.
Análisis semántico	¿La estructura satisface las restricciones de significado estático del lenguaje?	De una estructura reconocida a una estructura comprobada y, cuando corresponde, enriquecida con información.

2.1 Análisis léxico

El **analizador léxico** lee los caracteres del programa fuente y los agrupa en unidades que las fases posteriores pueden manejar. Estas unidades representan categorías como identificadores, números, palabras reservadas, operadores o signos de puntuación. Los espacios y los comentarios pueden separarse u omitirse cuando no son significativos para las siguientes fases.

Para el fragmento inicial, una salida léxica ilustrativa sería:

```
ID(valor) ASIG ID(valor) SUMA ID(inc) PYC
```

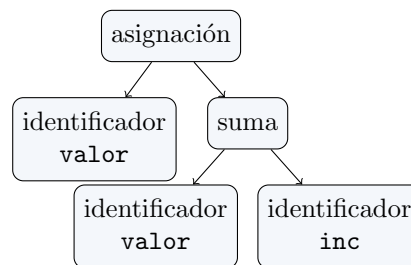
En esta representación, cada aparición de **ID** indica la categoría identificador y conserva el texto asociado. Los nombres **ASIG**, **SUMA** y **PYC** son etiquetas descriptivas usadas sólo en el ejemplo. Los conceptos de token, lexema y patrón, así como las reglas para reconocerlos, se estudiarán con detalle en otra nota.

La fase léxica todavía no decide si **valor** fue declarado ni si la suma es válida. Su responsabilidad en este punto es reconocer y clasificar las unidades del texto.

2.2 Análisis sintáctico

El **analizador sintáctico** recibe la secuencia producida por el analizador léxico y determina si sus elementos pueden agruparse según la gramática del lenguaje. Así reconoce construcciones mayores: expresiones, asignaciones, condicionales o ciclos, entre otras.

Una representación estructurada simplificada del fragmento puede verse así:



El árbol hace explícita una relación que en el texto sólo aparece por el orden de los símbolos: la suma de **valor** e **inc** produce el valor que se asignará. Los detalles de las gramáticas, las derivaciones y los árboles se estudiarán en la Unidad 3.

Las fuentes presentan variantes en la construcción de árboles. Algunas arquitecturas distinguen un árbol sintáctico y un ASA¹; otras hacen que el analizador sintáctico produzca directamente un ASA. Para el

¹ASA significa *árbol de sintaxis abstracta*. La denominación proviene del inglés *Abstract Syntax Tree*, abreviado AST.

compilador del curso, la decisión concreta deberá conservar una representación compatible con el verificador de tipos y el generador de código posteriores.

2.3 Análisis semántico

Una estructura puede ajustarse a la gramática y, aun así, incumplir otras reglas del lenguaje. El **analizador semántico** comprueba restricciones que no se resuelven únicamente con la forma sintáctica. Entre los ejemplos habituales se encuentran verificar que un identificador esté declarado, comprobar la compatibilidad de tipos y revisar que el uso de una función corresponda con su definición.

En el fragmento inicial, esta fase necesitaría saber a qué entidades se refieren `valor` e `inc` y qué tipos tienen. Sólo con esa información podría comprobar si la suma y la asignación son válidas. La nota no supone declaraciones ni tipos concretos para esos nombres; señala la información que la fase requeriría.

El resultado puede ser la misma estructura acompañada de información adicional o una nueva representación. Lo importante es el contrato: las fases posteriores deben saber que se realizaron las comprobaciones previstas y deben recibir los datos necesarios para continuar.

Un diagnóstico pertenece a una fase

Cada fase puede encontrar una clase diferente de problema:

- ★ La fase **léxica** detecta una secuencia de caracteres que no puede clasificar de acuerdo con las reglas léxicas.
- ★ La fase **sintáctica** detecta una secuencia de tokens que no puede organizar de acuerdo con la gramática.
- ★ La fase **semántica** detecta una estructura sintáctica que incumple una restricción, por ejemplo, una regla de declaración o de tipos.

El compilador debe indicar la ubicación y la naturaleza del problema con la mayor claridad posible. No debe enviar a generación de código una representación que ya sabe que es inválida.

Observación 2. Qué construimos y qué automatiza Lark

En notas y clase utilizaremos Lark para implementar el reconocimiento léxico y sintáctico del IMP base. Las prácticas recibirán esas versiones funcionales y las extenderán para IMP++. En ambos casos, Lark aplicará las reglas especificadas, pero no decidirá las categorías ni la estructura del lenguaje. La verificación semántica se implementará explícitamente en PYTHON mediante recorridos sobre la representación construida.

3. Representaciones intermedias, optimización y generación de código

Después del análisis comienza el trabajo de síntesis. La representación validada debe acercarse de manera progresiva al lenguaje destino. Para separar las particularidades del lenguaje fuente de las del lenguaje destino, muchos compiladores emplean una o más representaciones intermedias.

Definición 2. (Representación intermedia)

Una **representación intermedia** (IR, por sus siglas en inglés) es una forma interna del programa utilizada para comunicar fases y facilitar su análisis o transformación antes de producir el código destino.

Una representación intermedia puede adoptar forma de árbol, grafo o secuencia lineal de operaciones. No existe una única IR apropiada para todos los compiladores. Su diseño depende de la información que deben comunicar las fases y de las transformaciones que se realizarán después.

En términos generales, la síntesis puede incluir las tareas siguientes:

- ★ **Generación de una representación intermedia.** Convierte la estructura comprobada en una forma conveniente para las transformaciones posteriores.
- ★ **Optimización.** Transforma una representación para mejorar alguna característica del programa, como el tiempo de ejecución o el espacio utilizado, sin cambiar su comportamiento definido.
- ★ **Generación de código.** Traduce la representación disponible a instrucciones o construcciones del lenguaje destino.



El diagrama señala un recorrido frecuente. No obliga a que todos los compiladores tengan exactamente una IR o un único optimizador. La optimización puede aparecer en distintos momentos, repetirse o no estar presente en un compilador sencillo. Tampoco se utiliza para corregir programas fuente inválidos: su entrada ya debe haber superado las comprobaciones correspondientes.

En el compilador del curso, la salida de la generación de código será C. Esto permite expresar la relación con la cadena completa de manera explícita:



La forma concreta del ASA, de la representación intermedia y del código C se definirá en las unidades correspondientes. En este momento basta reconocer la responsabilidad de cada transformación y la necesidad de conservar el significado del programa.

Observación 3. Optimizar no significa cambiar el resultado

Una transformación sólo es aceptable si conserva el comportamiento definido del programa. Producir código más corto o más rápido no compensa una traducción incorrecta. En este curso se reconocerá la función de la optimización sin introducir técnicas avanzadas fuera del alcance del temario.

4. Integración y comunicación entre las fases

Dividir el compilador permite concentrar cada componente en una responsabilidad. Sin embargo, la división sólo funciona si las fases acuerdan qué información reciben y qué producen. La salida de una fase se convierte en la entrada de otra; por ello, un cambio en una representación puede afectar a varios componentes posteriores.

El flujo general puede resumirse con los siguientes contratos:

Fase	Entrada principal	Salida o información comunicada
Análisis léxico	Caracteres del programa fuente.	Secuencia de tokens con la información necesaria para reconocerlos y ubicarlos.
Análisis sintáctico	Secuencia de tokens.	Representación estructurada de las construcciones reconocidas.
Análisis semántico	Estructura sintáctica e información de símbolos.	Representación comprobada y datos semánticos requeridos por fases posteriores.
Generación de IR	Representación comprobada.	Operaciones o estructuras internas adecuadas para transformar el programa.
Optimización	IR.	Representación equivalente con la mejora buscada.
Generación de código	Representación intermedia o estructura validada.	Programa escrito en el lenguaje destino.

Dos componentes apoyan a varias fases y no deben imaginarse como simples pasos aislados al final del proceso:

- ★ **Tabla de símbolos.** Organiza información asociada con nombres del programa, por ejemplo su clase, tipo o ubicación. Diferentes fases pueden crear, consultar o completar esa información.
- ★ **Gestión de errores.** Permite registrar la ubicación del problema, producir un diagnóstico y decidir si es posible continuar el análisis. Los errores pueden aparecer en varias fases.

La organización también suele describirse mediante dos grandes partes. La **parte frontal** (*front end*) se concentra en comprender el lenguaje fuente y producir una representación comprobada. La **parte posterior** (*back end*) se concentra en transformar esa representación y producir código para el lenguaje destino. Una IR bien definida ayuda a separar ambas responsabilidades y favorece una arquitectura modular.

Para el proyecto acumulativo del curso, la comunicación prevista puede resumirse así:



El esquema no fija todavía las estructuras de datos ni la sintaxis concreta. Establece la dirección de las dependencias. En cada fase, las notas y clases producirán primero una versión funcional para IMP; después, la práctica correspondiente extenderá esa base para IMP++ y conservará las interfaces que utilizarán las fases posteriores.

Mini-checkpoint. Seguir la información

Al observar cualquier fase del compilador, conviene responder cuatro preguntas:

1. ¿Qué representación recibe?
2. ¿Qué tarea realiza y qué errores puede detectar?
3. ¿Qué representación o información produce?
4. ¿Qué fase utilizará ese resultado?

Si alguna respuesta no está definida, la comunicación entre los componentes todavía necesita precisarse.

Conclusión

Un compilador transforma un programa fuente en un programa destino mediante una secuencia organizada de fases. El análisis léxico agrupa caracteres; el análisis sintáctico reconoce estructuras; y el análisis semántico comprueba restricciones que dependen del significado estático del programa. Después, las representaciones intermedias facilitan la separación entre el lenguaje fuente y el lenguaje destino, la optimización transforma el programa sin alterar su comportamiento y la generación de código produce la salida prevista.

La arquitectura no consiste sólo en enumerar fases. Cada componente necesita una responsabilidad, una entrada y una salida claras; además, varias fases comparten información mediante estructuras como la tabla de símbolos y colaboran en el diagnóstico de errores. En el compilador acumulativo del curso, este flujo llevará primero de IMP a C y después conservará el mismo recorrido para las extensiones de IMP++. El siguiente tema concentrará la atención en el primer componente: el analizador léxico y las unidades que reconoce.

Referencias

- [1] Thain, D. (2026). *Introduction to Compilers and Language Design* (2.^a ed.). University of Notre Dame.
- [2] Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson/Addison-Wesley.
- [3] Cooper, K. D. y Torczon, L. (2012). *Engineering a Compiler* (2.^a ed.). Morgan Kaufmann/Elsevier.
- [4] Vilar Torres, J. M. (2010). *Estructura de los compiladores e intérpretes*. Universitat Jaume I.